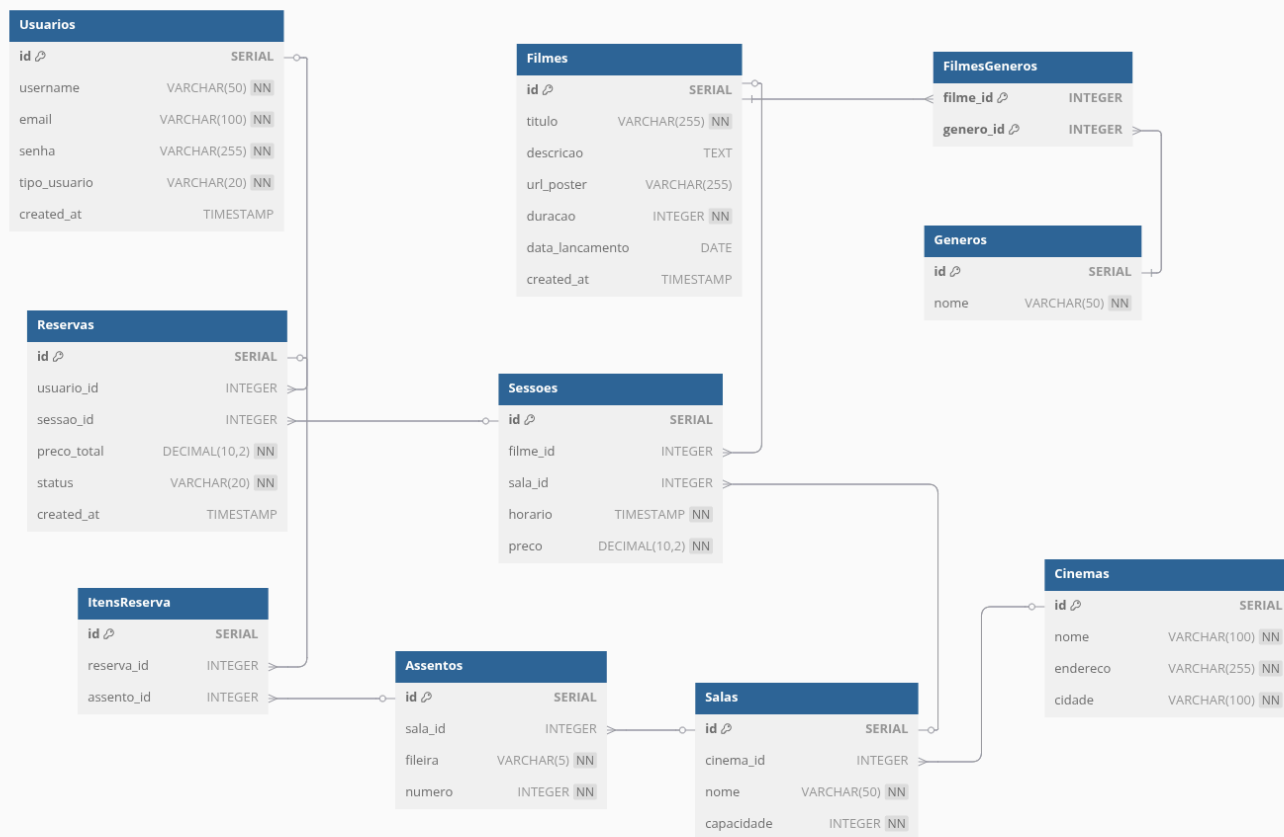


Você foi contratado para ajudar no desenvolvimento de um sistema de reserva de ingressos para cinemas, semelhante ao Ingresso.com. Este sistema permitirá gerenciar filmes, sessões, cinemas e reservas de assentos. Sua missão é demonstrar seu conhecimento e habilidades em Javascript ou Typescript, utilizando tecnologias como Express, Nest, Knex e Sequelize.

Considere que você faz parte de um time de desenvolvimento e as configurações de inicialização e conexões com o banco serão criadas por outro desenvolvedor. Portanto, não teremos que configurar package.json, tsconfig, nem configurar a conexão do Knex ou Sequelize.

O recorte do modelo do banco de dados a ser implementado pode ser visto a seguir:

- **Tabela Usuarios:** informações sobre os usuários do sistema, incluindo admins e usuários comuns
- **Tabela Filmes:** informações sobre os filmes disponíveis no sistema
- **Tabela Generos:** categorias para classificar os filmes
- **Tabela FilmesGeneros:** associação entre filmes e gêneros
- **Tabela Cinemas:** informações sobre os cinemas cadastrados no sistema
- **Tabela Salas:** informações sobre as salas de exibição em cada cinema
- **Tabela Assentos:** informações sobre os assentos disponíveis em cada sala
- **Tabela Sessoes:** informações sobre as sessões de cada filme (horários, sala, etc.)
- **Tabela Reservas:** informações sobre as reservas feitas pelos usuários
- **Tabela ItensReserva:** informações sobre os assentos específicos reservados em cada reserva



Questão 1: Faça a inicialização do seu framework ou gerenciador de requisições, descrevendo a criação de uma rota para listagem das sessões de filmes disponíveis para um determinado dia. Não é para criar as camadas do endpoint, apenas definir a rota do mesmo **[1 ponto]**.

Questão 2: Implemente um CRUD para a entidade de filmes, utilizando todas as camadas que foram apresentadas em sala de aula. Não é necessário acessar o banco de dados, mas a camada de acesso deve ser definida e utilizada, retornando apenas uma mensagem, ex: "Filme criado!" **[2 pontos]**.

- i) Listar filmes: **/filmes [GET]**
- ii) Visualizar filme: **/filme/id [GET]**
- iii) Criar filme: **/filme [POST]**
- iv) Atualizar filme: **/filme/id [PUT]**
- v) Deletar filme: **/filme/id [DELETE]**

Questão 3: Você precisa modificar o endpoint de criação de reserva e adicionar uma regra de negócio que só deixe criar uma reserva caso os assentos selecionados não estejam já reservados para aquela sessão. Lembre-se de criar os métodos necessários na camada de acesso ao banco de dados **[1 ponto]**.

Questão 4: Crie um endpoint completo, com todas as camadas necessárias para visualizar as informações de uma reserva. Esse endpoint deve retornar o título do filme, nome do cinema, sala onde a sessão acontecerá, o horário e os assentos reservados **[1 ponto]**.

Questão bônus: Crie um middleware `authMiddleware` para verificar se o usuário está autenticado via JWT. O token será enviado no cabeçalho `Authorization` como `Bearer <token>`. Se o token for inválido ou inexistente, a requisição deve ser rejeitada com erro 401 `Unauthorized`. Implemente esse middleware no endpoint de listar usuário. O corpo do endpoint não precisa ser implementado, apenas a assinatura e utilização do middleware **[1 ponto]**.

Boa prova!